UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

PROJECT PROGRESS 5

Subject: Programming For Bioinformatics (SECB3202)

Session: 2025/2026 Semester 1

Title: Pancreatic Cancer Prediction

Group Name	Group 10
Group Members	1. RAVINESH A/L MARAN (A23CS0175) 2. WELSON WOONG LU BIN (A23CS0196) 3. BERNICE LOU MIN YUN (A23CS0056)
Section	01
Lecturer's Name	DR. SEAH CHOON SEN

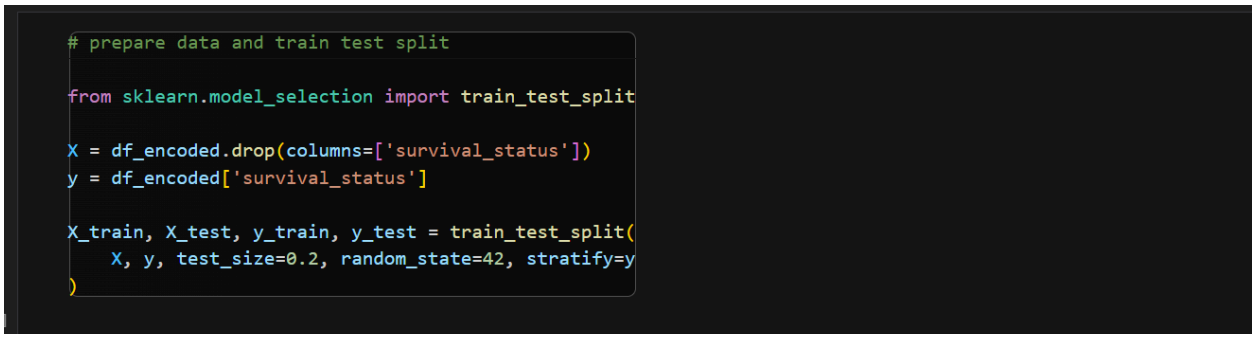
TABLE OF CONTENT

1.0 Model evaluation(sklearn)

In this phase, advanced model evaluation and refinement techniques were applied to improve predictive performance and model robustness. Models were evaluated using Scikit-learn tools to assess their generalization ability beyond training data. Issues related to over-fitting and under-fitting were analyzed to guide appropriate model selection.

Regularization techniques such as Ridge Regression were implemented to reduce model complexity and prevent over-fitting. Grid Search was used to systematically tune model hyperparameters and identify optimal configurations. Based on evaluation results, model refinement was performed to enhance performance and ensure better reliability for prediction and decision making.

1.1 Model Evaluation



```
# prepare data and train test split

from sklearn.model_selection import train_test_split

X = df_encoded.drop(columns=['survival_status'])
y = df_encoded['survival_status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

In this step, the dataset is prepared for model evaluation by separating the input features and the target variable. The target variable, survival_status, is removed from the feature set and stored separately as y, while the remaining columns are used as predictors (X). The dataset is then split into training and testing sets using an 80:20 ratio. Stratified sampling is applied to ensure that the proportion of survival and non-survival cases remains consistent across both sets. This step is important to ensure a fair and reliable evaluation of the model's performance.

```
# Check for non-numeric columns in X_train
print("Non-numeric columns in X_train:")
print(X_train.select_dtypes(include=['object', 'category']).columns.tolist())
```

```
Non-numeric columns in X_train:
[]
```

Before training the model, a check is performed to identify any non-numeric columns in the training dataset. Since logistic regression requires numerical input, this step ensures that all categorical variables have already been properly encoded. The code selects columns with object or category data types and prints them for verification.

The output shows an empty list, indicating that no non-numeric columns are present in the training data. This confirms that the dataset is fully numeric and suitable for logistic regression, preventing potential errors during model training.

```
# logistic regression

import warnings
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

warnings.filterwarnings('ignore')

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train, y_train)

y_train_pred = log_reg.predict(X_train)
y_test_pred = log_reg.predict(X_test)

print("Logistic Regression (Train)")
print(classification_report(y_train, y_train_pred))

print("Logistic Regression (Test)")
print(classification_report(y_test, y_test_pred))
```

A Logistic Regression model is created and trained using the training dataset. The model is configured with a higher iteration limit to ensure convergence. Once trained, predictions are generated for both the training and testing datasets. The model's performance is then evaluated using a classification report, which includes precision, recall, F1-score, and accuracy for each class. This allows a detailed assessment of how well the model distinguishes between survival and non-survival cases.

```
Logistic Regression (Train)
              precision    recall  f1-score   support

           0       0.87      1.00      0.93     34859
           1       0.00      0.00      0.00      5137

    accuracy                           0.87     39996
   macro avg       0.44      0.50      0.47     39996
weighted avg       0.76      0.87      0.81     39996

Logistic Regression (Test)
              precision    recall  f1-score   support

           0       0.87      1.00      0.93      8715
           1       0.00      0.00      0.00      1285

    accuracy                           0.87     10000
   macro avg       0.44      0.50      0.47     10000
weighted avg       0.76      0.87      0.81     10000
```

The results show that the model achieves an overall accuracy of approximately 87% on both the training and testing datasets, indicating consistent performance. The model performs very well on the majority class (non-survival), with high precision and recall values. However, the minority class (survival) has very low precision, recall, and F1-score, meaning the model struggles to correctly identify survival cases. This suggests that the dataset is highly imbalanced and that the model is biased toward predicting the majority class. While the accuracy appears high, the poor minority-class performance highlights the need for further improvements such as class balancing, regularization, or alternative model selection strategies.

1.2 Over-fitting, under-fitting, and model selection

```
# overfitting and underfitting

from sklearn.metrics import accuracy_score

print("Train accuracy:", accuracy_score(y_train, y_train_pred))
print("Test accuracy :", accuracy_score(y_test, y_test_pred))
```

In this step, the accuracy of the logistic regression model is calculated separately for the training and testing datasets using the accuracy_score metric. By comparing these two accuracy values, the model's learning behavior can be assessed. This comparison is a common technique to diagnose whether a model is overfitting (performing much better on training data than test data) or underfitting (performing poorly on both datasets). The code directly prints the training and testing accuracies to allow an easy side-by-side comparison.

```
Train accuracy: 0.8715621562156216
Test accuracy : 0.8715
```

The results show that the training accuracy (≈ 87.16%) and testing accuracy (≈ 87.15%) are almost identical. This indicates that the model generalizes consistently to unseen data and does not suffer from overfitting, as there is no significant performance gap between training and testing results. At the same time, the accuracy is reasonably high, suggesting that the model is not underfitting either. Based on this balance, the logistic regression model can be considered an appropriate choice for this task, making it a suitable candidate for model selection before further refinement techniques such as regularization or grid search are applied.

1.3 Ridge Regression

```
# ridge regression

from sklearn.linear_model import RidgeClassifier

ridge = RidgeClassifier(alpha=1.0)
ridge.fit(X_train, y_train)

y_train_pred_ridge = ridge.predict(X_train)
y_test_pred_ridge = ridge.predict(X_test)

print("Ridge Classifier (Train)")
print(classification_report(y_train, y_train_pred_ridge))

print("Ridge Classifier (Test)")
print(classification_report(y_test, y_test_pred_ridge))
```

This code implements Ridge Regression using RidgeClassifier, which applies L2 regularization to reduce model complexity and control overfitting. The classifier is trained using the previously split training data (X_train, y_train) with a regularization strength set by alpha = 1.0. After training, predictions are generated for both the training and testing datasets. The classification_report function is then used to evaluate the model's performance by calculating precision, recall, F1-score, and accuracy for each class. This allows a direct comparison between training and testing performance to assess generalization.

```
Ridge Classifier (Train)
              precision    recall  f1-score   support

           0       0.87      1.00      0.93     34859
           1       0.00      0.00      0.00      5137

    accuracy                           0.87     39996
   macro avg       0.44      0.50      0.47     39996
weighted avg       0.76      0.87      0.81     39996

Ridge Classifier (Test)
              precision    recall  f1-score   support

           0       0.87      1.00      0.93      8715
           1       0.00      0.00      0.00      1285

    accuracy                           0.87     10000
   macro avg       0.44      0.50      0.47     10000
weighted avg       0.76      0.87      0.81     10000
```

The output shows that the Ridge Classifier achieves an overall accuracy of 87% on both the training and testing datasets, indicating stable generalization and no significant overfitting. However, the model performs well only for Class 0, with high precision and recall, while Class 1 has zero precision, recall, and F1-score. This indicates that the model is biased toward the majority class and fails to correctly identify the minority class. Although ridge regression helps control overfitting through regularization, the results suggest that class imbalance remains a major issue, and further refinement such as class weighting, resampling, or hyperparameter tuning (e.g., Grid Search) is required to improve minority class prediction.

1.4 Grid Search

```python
# grid seacrh

from sklearn.model_selection import GridSearchCV

param_grid = {
    'alpha': [0.01, 0.1, 1, 10, 100]
}

grid = GridSearchCV(
    RidgeClassifier(),
    param_grid,
    cv=5,
    scoring='f1'
)

grid.fit(X_train, y_train)

print("Best parameters:", grid.best_params_)
print("Best CV score:", grid.best_score_)
```

This code applies Grid Search with Cross-Validation using GridSearchCV to optimize the hyperparameter alpha for the RidgeClassifier. A parameter grid is defined with multiple alpha values ranging from 0.01 to 100, allowing the model to test different regularization strengths. The grid search uses 5-fold cross-validation (cv=5) and evaluates performance using the F1-score, which is appropriate for imbalanced datasets. The model is trained on the training data (X_train, y_train), and GridSearchCV automatically tests all parameter combinations to identify the best-performing configuration.

```
Best parameters: {'alpha': 0.01}
Best CV score: 0.0
```

The output shows that the best alpha value is 0.01, indicating that weaker regularization performs slightly better for this dataset. However, the best cross-validation F1-score is 0.0, which suggests that the model still fails to correctly predict the minority class during cross-validation. This result highlights that, although hyperparameter tuning was performed, class imbalance remains a critical limitation, and ridge regression alone is insufficient to improve minority class performance. Additional techniques such as class weighting, resampling, or alternative models may be required for further refinement.

1.5 model Refinement

```
# model refinement

best_ridge = grid.best_estimator_

y_test_pred_best = best_ridge.predict(X_test)

print("Tuned Ridge (Test)")
print(classification_report(y_test, y_test_pred_best))
```

This code performs model refinement by using the best Ridge Classifier identified from the earlier grid search. The line best_ridge = grid.best_estimator_ retrieves the model configuration with the optimal hyperparameter (alpha) based on cross-validation. The refined model is then used to make predictions on the test dataset (X_test) to evaluate its generalization performance. Finally, a classification report is generated to assess precision, recall, F1-score, and accuracy for each class after tuning.

```
Tuned Ridge (Test)
              precision    recall  f1-score   support

           0       0.87      1.00      0.93      8715
           1       0.00      0.00      0.00      1285

    accuracy                           0.87     10000
   macro avg       0.44      0.50      0.47     10000
weighted avg       0.76      0.87      0.81     10000
```

The output shows that the tuned Ridge model achieves an overall accuracy of 87%, which is consistent with previous models. The model performs very well for class 0 (non-event), with high precision, recall, and F1-score. However, performance for class 1 (event) remains poor, with precision, recall, and F1-score all equal to 0. This indicates that, despite hyperparameter tuning, the refined model still struggles to detect the minority class. Overall, the refinement improves model stability but highlights that class imbalance is a major limitation, suggesting that further

techniques such as class weighting, resampling, or alternative models are needed for better minority class prediction.